

BUCHTAUSCH- APP DATENBANK

12 ENTITÄTEN, 3
TERNÄRE BEZIEHUNGEN,
SQL-STRUKTUR,
TESTFÄLLE

AGENDA

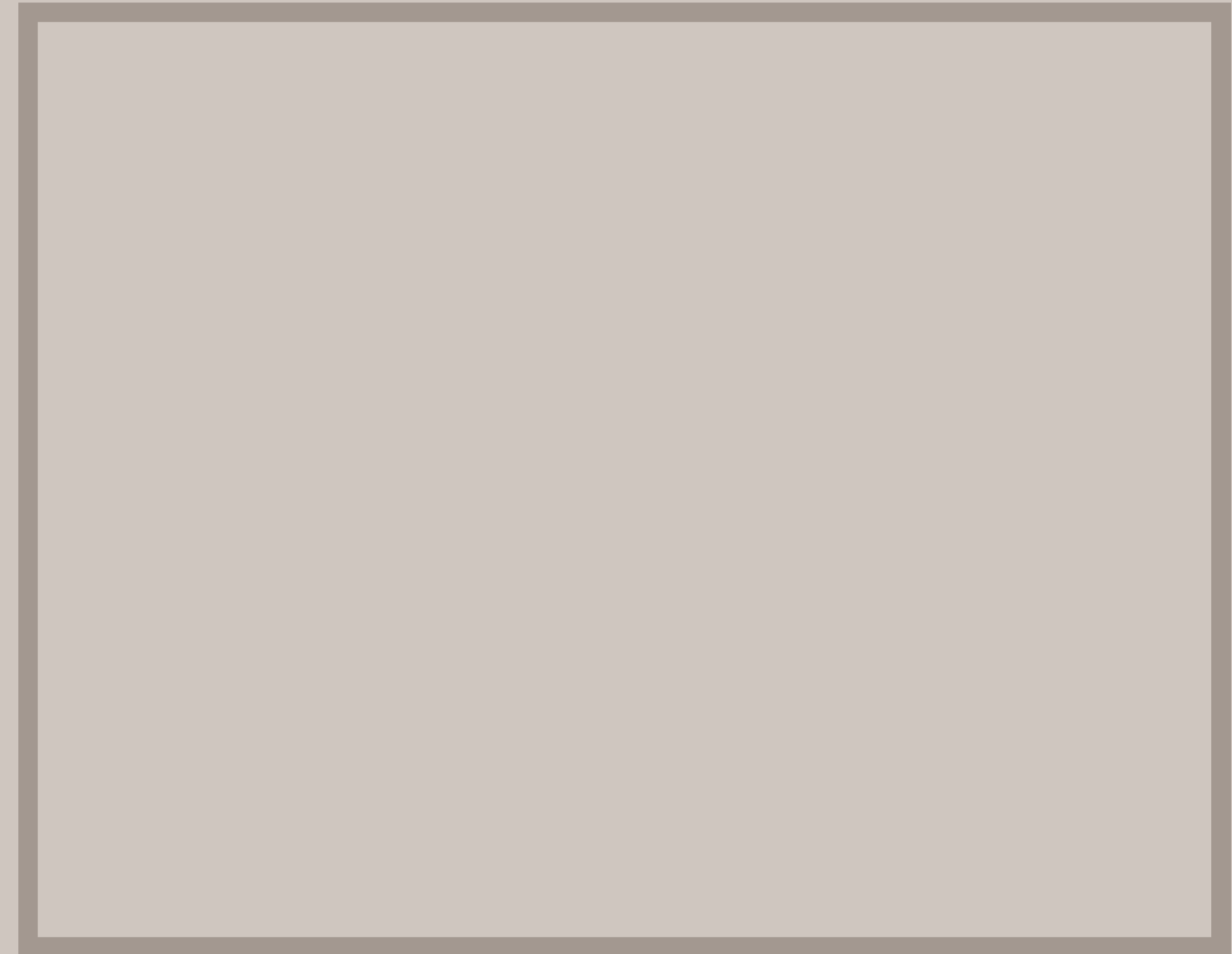
Datenbank-Überblick & ER-
Modell

Tabellen im Detail (12
Entitäten)

Ternäre Beziehungen (3 Stück)

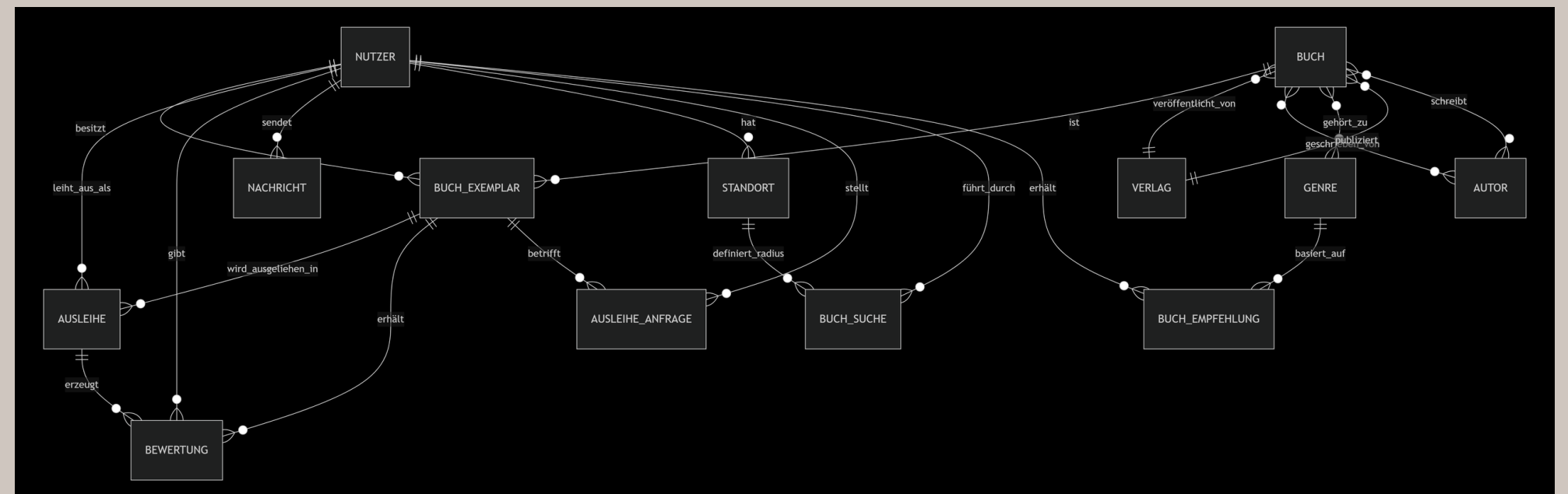
Testfälle & Ergebnisse

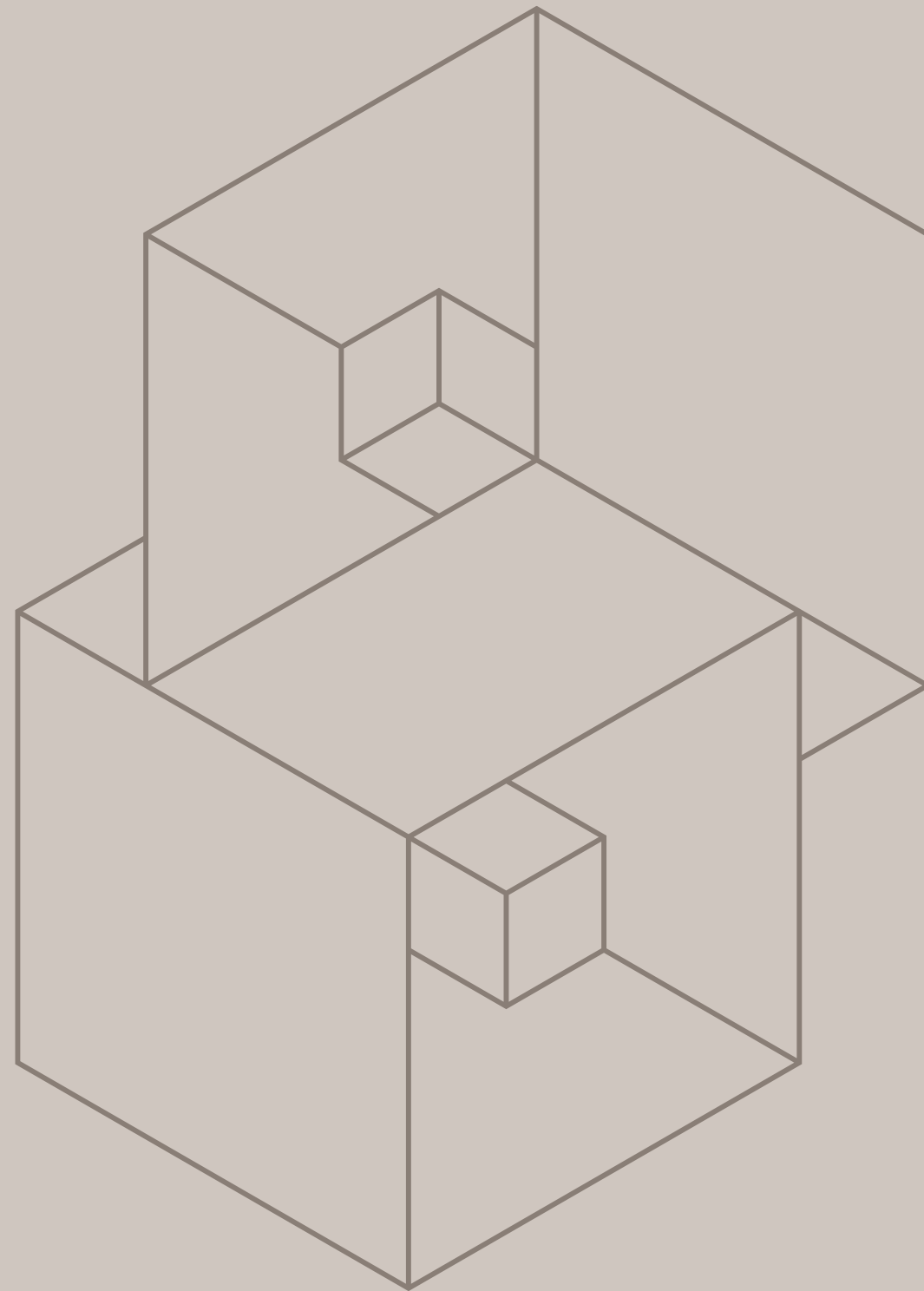
Zusammenfassung & Ausblick



ER-MODELL ÜBERSICHT

Darstellung des vollständigen Entity-Relationship-Modells mit allen 12 Entitäten und 3 ternären Beziehungen. Die Kardinalitäten (1:N, M:N, ternär) verdeutlichen die komplexen Datenbeziehungen der Buchtausch-Plattform.





ENTITÄT NUTZER

```
CREATE TABLE nutzer (  
  nutzer_id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  passwort_hash VARCHAR(255) NOT NULL,  
  vorname VARCHAR(100) NOT NULL,  
  nachname VARCHAR(100) NOT NULL,  
  adresse TEXT,  
  telefon VARCHAR(20),  
  registrierungsdatum DATE DEFAULT  
  CURRENT_DATE,  
  ist_aktiv BOOLEAN DEFAULT TRUE  
);
```

Besonderheiten: E-Mail-Validierung, aktiver Status
Rolle: Benutzer der Plattform, besitzt Exemplare,
tätigt Ausleihen

ENTITÄT VERLAG



```
CREATE TABLE verlag (  
verlag_id SERIAL PRIMARY KEY,  
name VARCHAR(200) UNIQUE NOT NULL,  
ort VARCHAR(100),  
website VARCHAR(255),  
email VARCHAR(150),  
telefon VARCHAR(30),  
gruendungsjahr INTEGER CHECK (gruendungsjahr >= 1400)  
);
```

Besonderheit: Historische Prüfung – Gründungsjahr muss ab 1400 liegen.

Rolle: Speichert Verlagsinformationen für die Zuordnung zu Büchern.

ENTITÄT AUTOR

```
CREATE TABLE autor (  
  autor_id SERIAL PRIMARY KEY,  
  vorname VARCHAR(100) NOT NULL,  
  nachname VARCHAR(100) NOT NULL,  
  geburtsdatum DATE CHECK (geburtsdatum <= CURRENT_DATE),  
  nationalitaet VARCHAR(50),  
  biografie TEXT,  
  website VARCHAR(255)  
);
```

Besonderheit: Geburtsdatum $\leq$ heute validiert

Rolle: Autoreninformationen für M:N Beziehung zu Büchern

ENTITÄT GENRE



```
CREATE TABLE genre (  
genre_id SERIAL PRIMARY KEY,  
name VARCHAR(100) UNIQUE NOT NULL,  
beschreibung TEXT,  
uebergeordnetes_genre_id INT REFERENCES genre(genre_id)  
);
```

Besonderheit: Hierarchische Genre-Struktur durch Selbstreferenz
Rolle: Kategorisierung von Büchern in Haupt- und Untergenres

ENTITÄT BUCH

```
CREATE TABLE buch (  
buch_id SERIAL PRIMARY KEY,  
isbn VARCHAR(17) UNIQUE NOT NULL,  
titel VARCHAR(255) NOT NULL,  
untertitel VARCHAR(255),  
verlag_id INTEGER REFERENCES verlag(verlag_id),  
erscheinungsjahr INTEGER CHECK  
(erscheinungsjahr >= 1400),  
sprache VARCHAR(50),  
seitenzahl INTEGER CHECK (seitenzahl > 0),  
beschreibung TEXT  
);
```

Besonderheiten: FK zu Verlag, ISBN-Eindeutigkeit,
Validierung Jahr/Seitenzahl

Rolle: Zentrale Buchinformationen der Plattform

BUCH_AUTOR (M:N BEZIEHUNG)



```
CREATE TABLE buch_autor (  
  buch_id INTEGER NOT NULL,  
  autor_id INTEGER NOT NULL,  
  hauptautor BOOLEAN DEFAULT FALSE,  
  PRIMARY KEY (buch_id, autor_id),  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id) ON DELETE CASCADE,  
  FOREIGN KEY (autor_id) REFERENCES autor(autor_id) ON DELETE CASCADE  
);
```

- Zusammengesetzter Primärschlüssel aus buch_id und autor_id
- ON DELETE CASCADE für referenzielle Integrität
- hauptautor-Flag kennzeichnet den Hauptautor bei Co-Autorenschaften
- Rolle: Verknüpfung von Büchern zu mehreren Autoren (M:N)



BUCH_GENRE (M:N BEZIEHUNG)

```
CREATE TABLE buch_genre (  
  buch_id INT NOT NULL,  
  genre_id INT NOT NULL,  
  PRIMARY KEY (buch_id, genre_id),  
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id) ON DELETE CASCADE,  
  FOREIGN KEY (genre_id) REFERENCES genre(genre_id) ON DELETE CASCADE  
);
```

Zusammengesetzter Primärschlüssel aus buch_id und genre_id. ON DELETE CASCADE gewährleistet referentielle Integrität. Rolle: Verknüpfung von Büchern zu mehreren Genres.

ENTITÄT STANDORT

```
CREATE TABLE standort (  
  standort_id SERIAL PRIMARY KEY,  
  nutzer_id INTEGER REFERENCES nutzer (nutzer_id),  
  bezeichnung VARCHAR(100),  
  strasse VARCHAR(200),  
  plz VARCHAR(10),  
  stadt VARCHAR(100),  
  latitude DECIMAL(9,6) CHECK (latitude BETWEEN -90 AND 90),  
  longitude DECIMAL(9,6) CHECK (longitude BETWEEN -180 AND 180),  
  abholzeiten TEXT,  
  versand_option BOOLEAN DEFAULT FALSE,  
  ist_standard BOOLEAN DEFAULT FALSE  
);
```

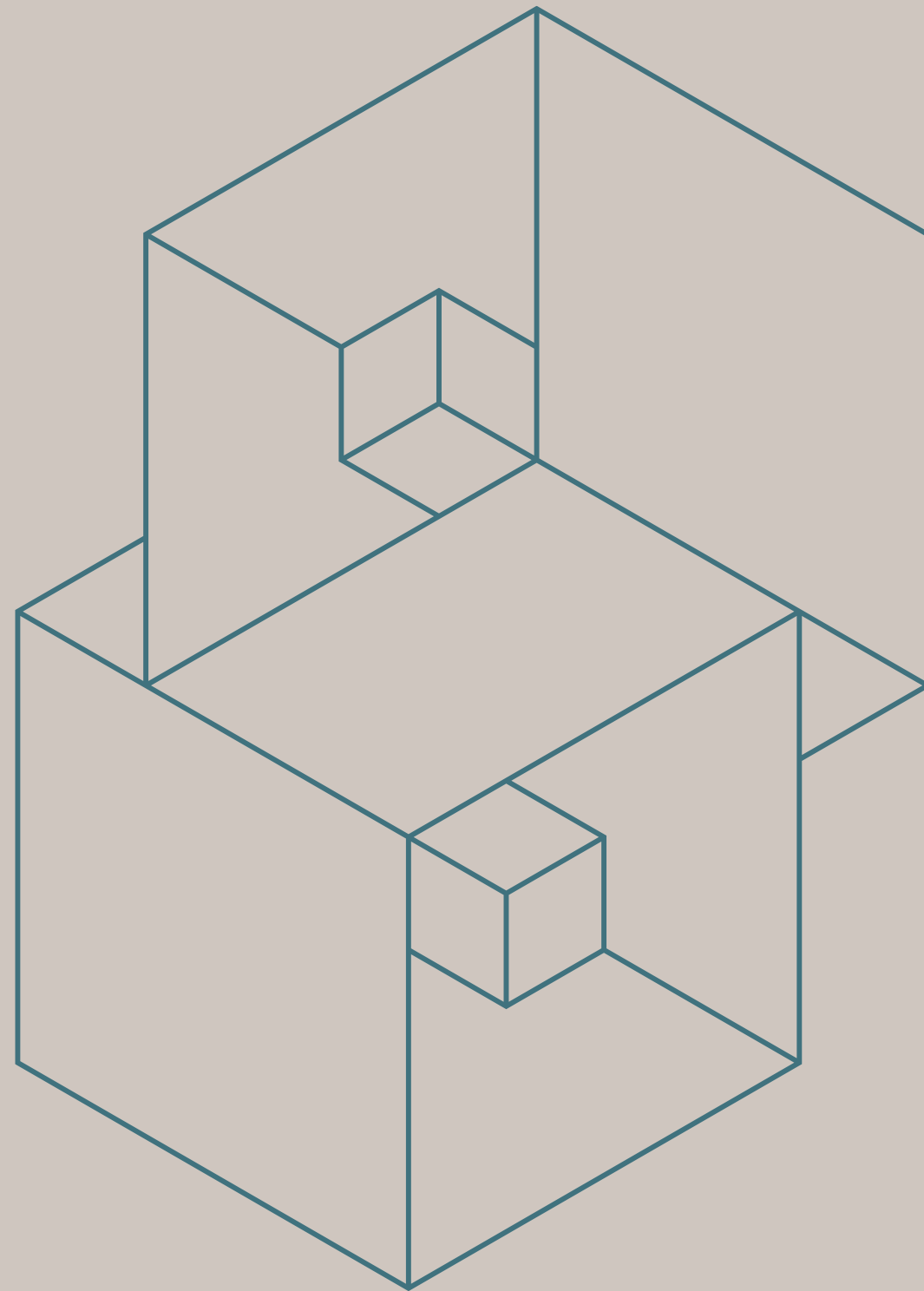
Besonderheiten: Koordinatenvvalidierung für Geo-Suche, Versandoption als Alternative zur Abholung.

Rolle: Definiert Buchübergabe- und Suchorte der Nutzer.

ENTITÄT BUCH_EXEMPLAR

```
CREATE TABLE buch_exemplar (  
  exemplar_id SERIAL PRIMARY KEY,  
  buch_id INTEGER REFERENCES buch(buch_id),  
  besitzer_id INTEGER REFERENCES nutzer(nutzer_id),  
  standort_id INTEGER REFERENCES standort(standort_id),  
  zustand VARCHAR(50) CHECK (zustand IN ('neu', 'sehr gut', 'gut', 'akzeptabel', 'schlecht')),  
  status VARCHAR(20) CHECK (status IN ('verfügbar', 'ausgeliehen', 'reserviert')),  
  max_leihdauer_tage INTEGER DEFAULT 30,  
  notizen TEXT,  
  anzahl_ausleihen INTEGER DEFAULT 0  
);
```

Rolle: Physische Buchexemplare mit Besitz, Standort und Verfügbarkeitsstatus.



ENTITÄT AUSLEIHE

```
CREATE TABLE ausleihe (  
  ausleihe_id SERIAL PRIMARY KEY,  
  exemplar_id INT REFERENCES  
    buch_exemplar(exemplar_id),  
  verleih_id INT REFERENCES nutzer(nutzer_id),  
  entleiher_id INT REFERENCES nutzer(nutzer_id),  
  standort_id INT REFERENCES standort(standort_id),  
  startdatum DATE NOT NULL,  
  enddatum DATE NOT NULL,  
  rueckgabedatum DATE,  
  status VARCHAR(20) CHECK (status IN ('aktiv',  
    'abgeschlossen', 'ueberfaellig'))  
);
```

Rolle: Dokumentation aller Ausleihvorgänge zwischen Nutzern.

AUSLEIHE_ANFRAGE



```
CREATE TABLE ausleihe_anfrage (  
  anfrage_id SERIAL PRIMARY KEY,  
  nutzer_id INT NOT NULL REFERENCES nutzer(nutzer_id),  
  exemplar_id INT NOT NULL REFERENCES buch_exemplar(exemplar_id),  
  gewuenschter_start DATE NOT NULL,  
  gewuensches_ende DATE NOT NULL,  
  status VARCHAR(20) DEFAULT 'offen',  
  nachricht TEXT,  
  anfragedatum TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Ternäre Beziehung: Nutzer → Exemplar → Zeitraum

Status-Constraints: offen, angenommen, abgelehnt, storniert

ENTITÄT BEWERTUNG

```
CREATE TABLE bewertung (  
  bewertung_id SERIAL PRIMARY KEY,  
  bewertende_id INT NOT NULL REFERENCES  
  nutzer(nutzer_id),  
  bewertete_id INT REFERENCES nutzer(nutzer_id),  
  exemplar_id INT REFERENCES buch_exemplar(exemplar_id),  
  ausleihe_id INT REFERENCES ausleihe(ausleihe_id),  
  sterne INT NOT NULL CHECK (sterne BETWEEN 1 AND 5),  
  kommentar TEXT,  
  datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Besonderheit: XOR-Bedingung – Bewertung entweder für
Nutzer ODER Exemplar, nicht beides gleichzeitig.
Sternebewertung 1-5 validiert.

ENTITÄT NACHRICHT

```
CREATE TABLE nachricht (  
  nachricht_id SERIAL PRIMARY KEY,  
  sender_id INT NOT NULL REFERENCES nutzer(nutzer_id),  
  empfaenger_id INT NOT NULL REFERENCES nutzer(nutzer_id),  
  betreff VARCHAR(255),  
  inhalt TEXT NOT NULL,  
  zeitstempel TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  gelesen BOOLEAN DEFAULT FALSE  
);
```

Rolle: Ermöglicht die direkte Kommunikation zwischen Nutzern der Plattform für Absprachen zu Ausleihen und Übergaben.

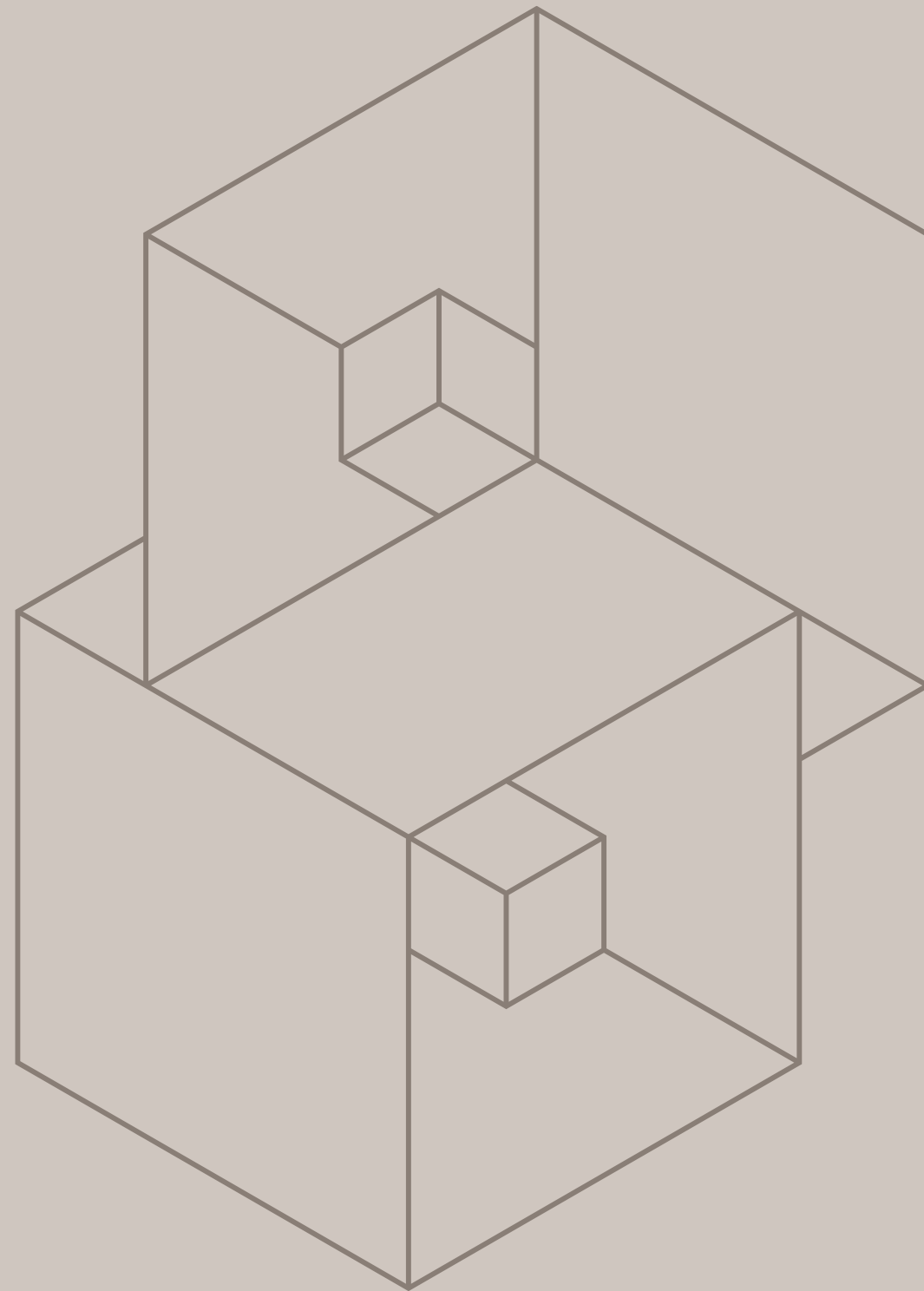
BUCH_SUCHE (TERNÄRE BEZIEHUNG)

```
CREATE TABLE buch_suche (  
  suche_id SERIAL PRIMARY KEY,  
  nutzer_id INT REFERENCES nutzer(nutzer_id),  
  standort_id INT REFERENCES standort(standort_id),  
  genre_id INT REFERENCES genre(genre_id),  
  suchradius_km DECIMAL(5,2),  
  suchbegriff VARCHAR(255),  
  suchzeitpunkt TIMESTAMP DEFAULT NOW(),  
  ergebnis_anzahl INT DEFAULT 0  
);
```

Ternäre Beziehung: Nutzer × Standort × Genre

Erfassung von Suchanfragen mit räumlicher Komponente

Rolle: Analyse von Suchverhalten und Nutzerinteressen

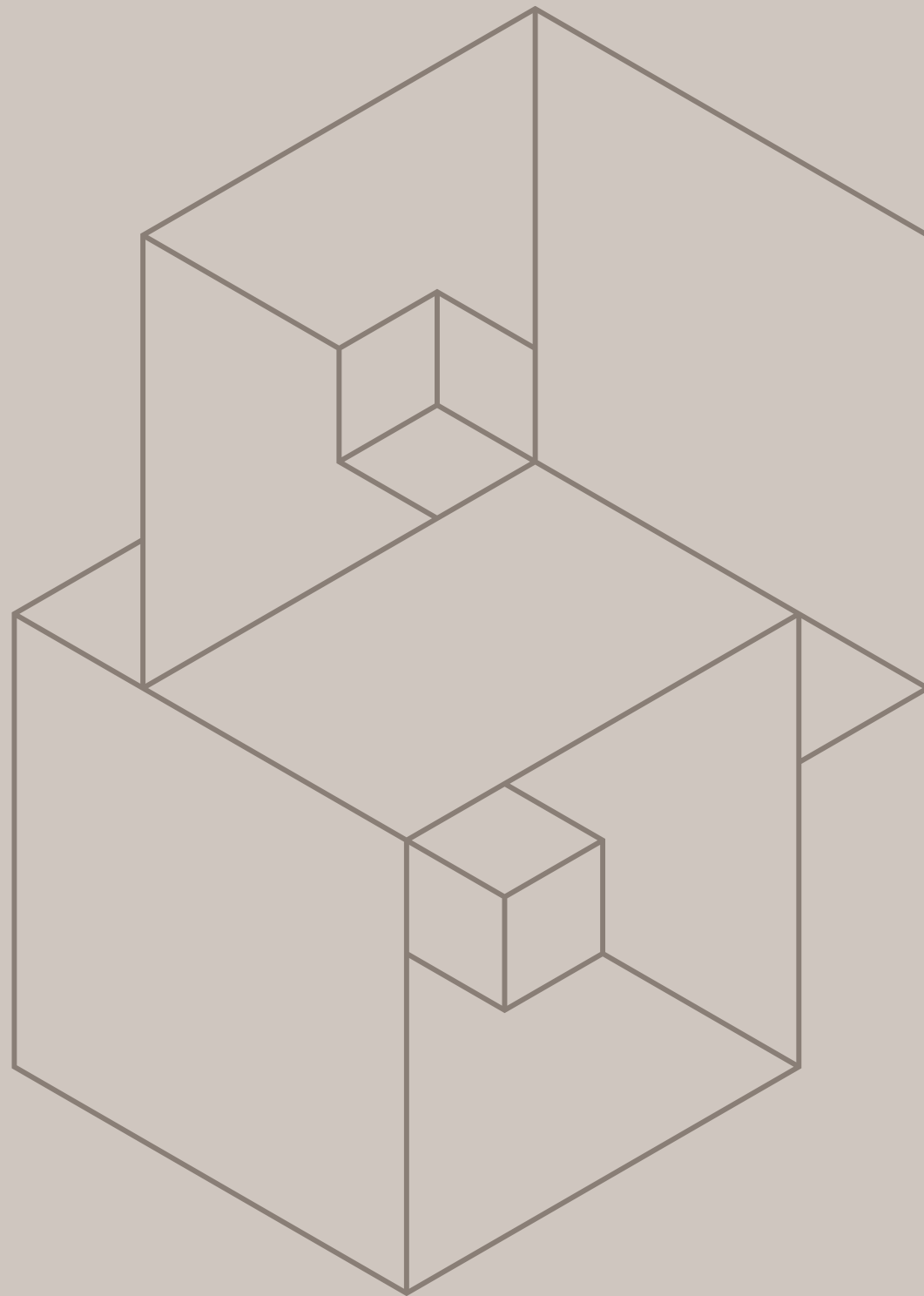


TESTFALL 1 – VERFÜGBARE BÜCHER IN BERLIN

SQL-Query zur Abfrage verfügbarer Bücher mit Standort Berlin:

```
SELECT b.titel, b.isbn, n.vorname || ' ' || n.nachname  
AS besitzer,  
s.strasse, s.plz, s.stadt, be.zustand,  
be.max_leihdauer_tage  
FROM buch_exemplar be  
JOIN buch b ON be.buch_id = b.buch_id  
JOIN nutzer n ON be.besitzer_id = n.nutzer_id  
JOIN standort s ON be.standort_id = s.standort_id  
WHERE be.status = 'verfuegbar' AND s.stadt =  
'Berlin';
```

Erwartetes Ergebnis: Liste aller verfügbaren Exemplare in Berlin mit Besitzer- und Standortdaten.



TESTFALL 2 – AKTIVE AUSLEIHEN MIT FÄLLIGKEIT

SQL-Query zur Abfrage aktiver und abgeschlossener Ausleihen mit Statusanzeige:

```
SELECT a.ausleihe_id, b.titel, n.vorname || ' ' || n.nachname  
AS entleiher,  
a.startdatum, a.enddatum, a.rueckgabedatum,  
CASE WHEN a.rueckgabedatum IS NOT NULL THEN  
'Zurückgegeben'  
WHEN a.enddatum < CURRENT_DATE THEN 'Überfällig'  
ELSE 'Noch ausgeliehen' END AS status  
FROM ausleihe a  
JOIN buch_exemplar be ON a.exemplar_id =  
be.exemplar_id  
JOIN buch b ON be.buch_id = b.buch_id  
JOIN nutzer n ON a.entleiher_id = n.nutzer_id;  
Erwartetes Ergebnis: Übersicht aller Ausleihen mit Datum  
und aktuellem Status.
```

TESTFALL 3 – TOP BEWERTETE BÜCHER

SQL-Query zur Ermittlung der bestbewerteten Bücher:

```
SELECT b.titel, b.isbn, ROUND(AVG(bw.sterne), 2) AS  
durchschnitt, COUNT(bw.bewertung_id) AS anzahl_bewertungen  
FROM buch b
```

```
JOIN buch_exemplar be ON b.buch_id = be.buch_id
```

```
JOIN bewertung bw ON be.exemplar_id = bw.exemplar_id
```

```
GROUP BY b.buch_id, b.titel, b.isbn
```

```
HAVING AVG(bw.sterne) >= 4.0
```

```
ORDER BY durchschnitt DESC, anzahl_bewertungen DESC;
```

Erwartetes Ergebnis: Liste aller Bücher mit

Durchschnittsbewertung $\geq 4,0$ Sterne, sortiert nach Bewertung
und Anzahl.

TESTFALL 4 – RÄUMLICHE SUCHE

SQL-Query: Suche nach verfügbaren Büchern im Umkreis von 10 km um München unter Verwendung der Haversine-Formel zur Entfernungsberechnung.

Erwartetes Ergebnis: Liste verfügbarer Exemplare mit Buchtitel, Standortbezeichnung, Stadt und berechneter Entfernung in Kilometern.

Besonderheiten: Nutzung von latitude/longitude aus der Standort-Tabelle, Filterung nach status = 'verfuegbar', Sortierung nach Entfernung aufsteigend.

TESTFALL 5 – NUTZER-STATISTIKEN

SQL-Query: Aggregierte Daten pro Nutzer mit Anzahl Bücher, Ausleihen, Bewertungen und Durchschnittsbewertung.

```
SELECT n.nutzer_id, n.vorname, n.nachname,  
COUNT(DISTINCT be.exemplar_id) AS anzahl_buecher,  
COUNT(DISTINCT a.ausleihe_id) AS anzahl_ausleihen,  
COUNT(DISTINCT b.bewertung_id) AS anzahl_bewertungen,  
ROUND(AVG(b.sterne), 2) AS durchschnitt_sterne  
FROM nutzer n  
LEFT JOIN buch_exemplar be ON n.nutzer_id = be.besitzer_id  
LEFT JOIN ausleihe a ON n.nutzer_id = a.entleiher_id  
LEFT JOIN bewertung b ON n.nutzer_id = b.bewertete_id  
GROUP BY n.nutzer_id;
```

Erwartetes Ergebnis: Übersicht der Nutzeraktivitäten mit allen relevanten Kennzahlen.

ZUSAMMENFASSUNG

- 12 Tabellen strikt in 3. Normalform implementiert
- 3 komplexe ternäre Beziehungen erfolgreich modelliert
- Umfassende Validierung der Datenintegrität durch Constraints
- CHECK-Constraints für Wertebereichsprüfungen

- Integration räumlicher Suchfunktion mit Koordinaten
- Bewertungssystem für Nutzer und Exemplare
 - Testfälle demonstrieren Funktionalität
 - Hohe Datenqualität durch referentielle Integrität

AUSBLICK PHASE 3



Aufbau eines GitHub-Repositories mit vollständiger Dokumentation und Versionierung aller Datenbankskripte und Testfälle
Erstellung eines zusammenfassenden PDFs mit detaillierter Performance-Analyse der implementierten Abfragen
Implementierung erweiterter Features: Benachrichtigungssystem, erweiterte Suchfunktionen und Skalierbarkeitsoptimierungen
Bereitstellung eines ZIP-Archivs mit allen relevanten Dateien für die finale Abgabe

VIELEN DANK!

Haben Sie Fragen? Kontaktdaten und GitHub-Link folgen in
Phase 3.

